

EPIC 09 - Task 03: API & Database Design

For the **Rapido/Bike Taxi System**, We have to create APIs for Authentication, Users, Drivers, Ride Booking, Ride Tracking, Payments, and Notifications.

1. API Design

Authentication APIs

Method	Endpoint	Purpose
POST	/api/auth/register	Register a user
POST	/api/auth/login	Login
POST	/api/auth/logout	Logout
POST	/api/auth/refresh-token	Refresh JWT token

Example:

POST /api/auth/login

```
{  
  "email": "user@gmail.com",  
  "password": "password123"  
}
```

User APIs

Method	Endpoint	Purpose
POST	/api/users	Create user
GET	/api/users/{id}	Get user
PUT	/api/users/{id}	Update user
DELETE	/api/users/{id}	Delete user
GET	/api/users/{id}/rides	Get user's ride history

Driver APIs

Method	Endpoint	Purpose
POST	/api/drivers	Register driver
GET	/api/drivers/{id}	Get driver details
PUT	/api/drivers/{id}	Update driver
PUT	/api/drivers/{id}/status	Update online/offline status
GET	/api/drivers/available	Find available drivers

Example:

PUT /api/drivers/101/status

```
{  
  "status": "ONLINE"  
}
```

2. Ride Booking APIs

Method	Endpoint	Purpose
POST	/api/rides	Book a ride
GET	/api/rides/{id}	Get ride details
PUT	/api/rides/{id}/accept	Driver accepts ride
PUT	/api/rides/{id}/start	Start ride
PUT	/api/rides/{id}/complete	Complete ride
PUT	/api/rides/{id}/cancel	Cancel ride
GET	/api/rides/user/{userId}	User's rides

Example booking request:

POST /api/rides

```
{  
  "userId": 101,  
  "pickupLocation": "Hitech City",  
  "dropLocation": "Banjara Hills"  
}
```

Response:

```
{
  "rideId": 5001,
  "status": "REQUESTED",
  "estimatedFare": 250.00
}
```

3. Ride Tracking APIs

Method	Endpoint	Purpose
GET	/api/rides/{rideId}/location	Get current driver location
POST	/api/location/update	Update driver location
GET	/api/rides/{rideId}/tracking	Track ride

Example:

POST /api/location/update

```
{
  "driverId": 201,
  "latitude": 17.4483,
  "longitude": 78.3915
}
```

For real-time tracking, **WebSocket** can be used in addition to REST APIs.

4. Payment APIs

Method	Endpoint	Purpose
POST	/api/payments	Make payment
GET	/api/payments/{id}	Get payment
POST	/api/payments/{id}/refund	Refund payment
GET	/api/payments/user/{userId}	Payment history

Example:

POST /api/payments

```
{  
  "rideId": 5001,  
  "userId": 101,  
  "amount": 250.00,  
  "paymentMethod": "UPI"  
}
```

5. Notification APIs

Method	Endpoint	Purpose
POST	/api/notifications	Send notification
GET	/api/notifications/{userId}	Get notifications
PUT	/api/notifications/{id}/read	Mark notification as read

Notifications can be sent for:

- Ride booked
- Driver assigned
- Driver arriving
- Ride started
- Ride completed
- Payment successful
- Ride cancelled

6. Database Design

Each microservice should preferably **own its own database/schema**.

User Database

USERS

id PK
name
email
phone
password
role
created_at
updated_at

Driver Database

DRIVERS

id PK
name
phone
email
license_number
vehicle_number
vehicle_type
status
created_at

Ride Database

RIDES

id PK
user_id
driver_id
pickup_location
drop_location
pickup_latitude
pickup_longitude

drop_latitude
drop_longitude
fare
status
created_at
completed_at

Payment Database

PAYMENTS

id PK
ride_id
user_id
amount
payment_method
transaction_id
status
created_at

Location Database

DRIVER_LOCATIONS

id PK
driver_id
latitude
longitude
updated_at

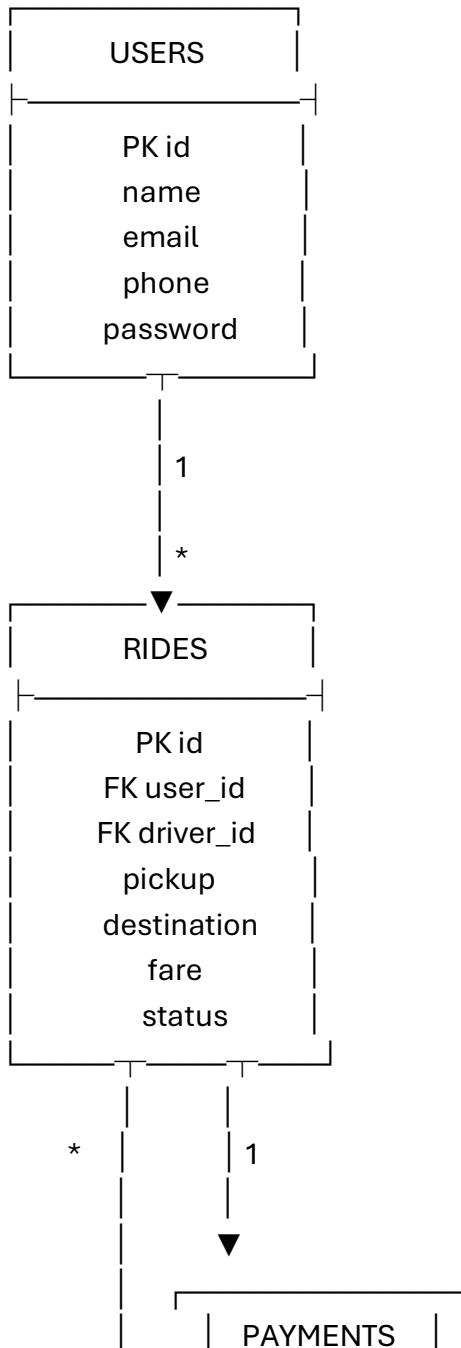
Notification Database

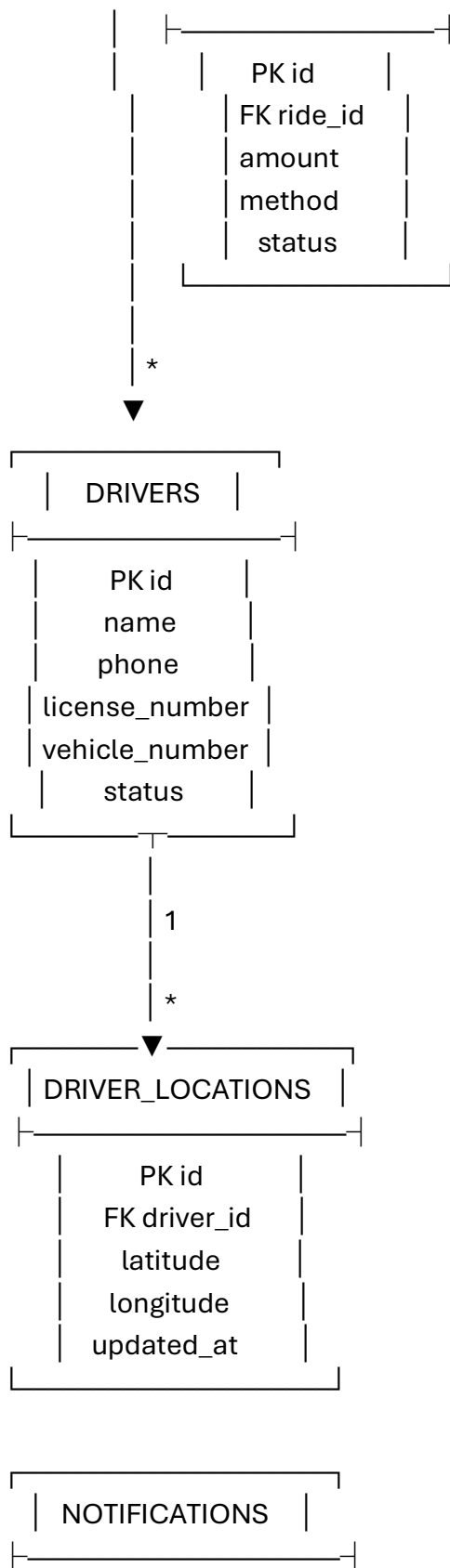
NOTIFICATIONS

id PK
user_id
ride_id
message

type
status
created_at

7. ER Diagram





PK id
FK user_id
FK ride_id
message
type
status

8. Swagger / OpenAPI

Document all APIs using **Swagger/OpenAPI**.

For Spring Boot, add:

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.8.13</version>
</dependency>
```

After starting the application, Swagger UI can be accessed at:

`/swagger-ui/index.html`